Informe final de pruebas de software: auditoría del coordinador, gestión de profesores y accesibilidad en EntreAulas

Juliana Franco

Facultad de Ingeniería, Universidad de Medellín

Pruebas de software

11 de noviembre de 2026

Autora: Juliana Franco · **Curso:** Pruebas de software · **Universidad de Medellín** · **Fecha:** 11 de noviembre de 2026 **Repositorios:** `EntreAulas_Front` y `EntreAulas_Back`, rama `pruebas-final-juliana` (commits locales; no se hizo *push*).

Resumen

Se probó un conjunto de tres requisitos nuevos de EntreAulas (siete funcionalidades): la vista de auditoría del coordinador, el rediseño de la gestión de profesores y las mejoras de accesibilidad (modo noche, español/inglés, tamaño de letra estándar, dashboard con barra lateral por rol y calendario de evaluaciones). Se aplicaron TDD, caja blanca y negra, pruebas unitarias, de componente, de API, E2E con Cypress y Playwright, BDD con Screenplay, accesibilidad con axe-core, seguridad según OWASP Top 10 (Open Worldwide Application Security Project [OWASP], 2021), análisis estático con SonarQube y evaluación de IA con Playwright, DeepEval y Stagehand. Se ejecutaron 1 115 pruebas en el back y 774 en el front (88 y 177 nuevas), 70 pruebas E2E de Cypress y 25 de Playwright; todas pasan, tanto en desarrollo como sobre el despliegue en Docker. Se registraron 10 defectos (7 corregidos, 3 abiertos), entre ellos la baja precisión (0.455) y exhaustividad (0.5) del detector de acoso de la IA. SonarQube dio Quality Gate OK en ambos repositorios (cobertura 94.6 % y 95.3 %). La evaluación con usuarios (SUS) quedó preparada pero pendiente de participantes.

1. Introducción y alcance

EntreAulas es una aplicación web de evaluación docente (React + Express + Supabase). El alcance individual se describe en `01-PRESENTACION-Y-PLAN.md`: F1 auditoría, F2 gestión de profesores, F3 modo noche, F4 idioma, F5 tamaño de letra, F6 dashboard con barra lateral y F7 calendario. Quedan fuera el inicio de sesión y el CRUD de usuarios.

2. Metodología

Se siguió la pirámide de pruebas y la terminología ISTQB (International Software Testing Qualifications Board [ISTQB], 2023). La auditoría se construyó con TDD en tres fases —rojo, verde y refactor— (Beck, 2003), con patrón Arrange-Act-Assert, principios FIRST y dobles de prueba (*fake*, *stub*, *spy*, *mock*). La complejidad ciclomática se calculó como V(G) = decisiones + 1 (McCabe, 1976) con un script propio. La accesibilidad se evaluó contra WCAG 2.1 (World Wide Web Consortium [W3C], 2018) con axe-core y comprobaciones manuales; la usabilidad con las diez heurísticas de Nielsen (1994) y, pendiente, el SUS (Brooke, 1996). El diseño visual se documentó con el criterio *frontend-design*. La matriz de trazabilidad usa el formato de la clase (37 casos, 11 escenarios).

3. Resultados

3.1 Ejecución y cobertura

Nivel	**Resultado**
Back (Vitest + Supertest)	1 115 ✔ (88 nuevas) · cobertura del código nuevo 97.6 % líneas
Front (Vitest + Testing Library)	774 ✔ (177 nuevas) · 94.7 % líneas, 82.6 % ramas
Cypress: UI / accesibilidad / seguridad / API / BDD	36 / 13 / 8 / 7 / 6 = 70 ✔
Playwright (agentes)	25 ✔ (4 iteraciones)
Despliegue en Docker	Cypress 64/64 y Playwright 25/25
SonarQube	Quality Gate OK; 0 bugs, 0 vulnerabilidades; code smells 12 (back) y 15 (front); duplicación 0 % y 5.3 %

Tras la revisión visual con la usuaria se hicieron dos ajustes finales de interfaz, con su prueba y commit: la barra lateral clara en modo claro y ocultable con un botón de flecha (recordado en el navegador), y la distribución del dashboard con las métricas apiladas a la izquierda y el calendario a la derecha en los cinco roles (caso CP-DSH-05).

3.2 Caja blanca y negra

Ninguna función supera V(G) = 9; se cubrieron todos los caminos independientes de `leerAccion` (5), `parsearFiltrosAuditoria` (6), `estadoDelDocente` (4), `rutaActiva` (6) y otras. Con partición de equivalencia y valores límite se probaron `page`, `pageSize` (0, 1, 50, 51), fechas, acciones y el umbral de riesgo (3.99, 4.0). Detalle y diagramas en `CAJA-BLANCA-Y-NEGRA.md`.

3.3 Defectos

ID	Sev.	Resumen	Estado
DEF-AUD-01	Baja	El error 400 reflejaba la entrada del usuario	Corregido
DEF-REDI-00	Media	Error del calendario como `alert` rompía la regresión del decano	Corregido
DEF-REDI-01	Alta	Menú lateral cerrado enfocable por teclado	Corregido
DEF-REDI-02/04	Media/Baja	Desbordes horizontales (reflujo)	Corregido
DEF-REDI-03	Alta	axe: grid sin filas, botón sin nombre, contraste	Corregido
DEF-REDI-05	Media	La paginación volvía a la página 1	Corregido
DEF-IA-01/02	Alta/Media	Detector de acoso: recall 0.5 y precisión 0.455	Abierto
Heredados (11)	Media	Pruebas Cypress obsoletas, idénticas en la línea base	Abierto

3.4 Regresión

Contra la rama base (`testing`) y la rama final, la suite previa de Cypress (142 pruebas) dio el mismo resultado (121 ✔, 11 ✘, 10 omitidas): no hay regresiones nuevas. Tras `npm audit fix` y cada corrección se repitieron las suites completas.

3.5 Accesibilidad, usabilidad y seguridad

Los criterios WCAG 1.3.1, 1.4.3, 1.4.4, 1.4.10, 2.1.1, 2.4.1, 2.4.3, 2.4.7, 3.1.1, 3.3.1, 4.1.2 y 4.1.3 quedaron cumplidos tras corregir cinco barreras. La heurística de Nielsen halló solo problemas leves (consistencia de idioma en otras pantallas). Las pruebas con usuarios y el SUS no se ejecutaron por falta de participantes. En seguridad, los controles OWASP A01–A05, A08 y A09 pasan; las dependencias pasaron de 13 a 7 vulnerabilidades y quedan cuatro sin parche; no se pudo ejecutar un escaneo dinámico con OWASP ZAP por la red de Docker del equipo.

3.6 Inteligencia artificial

Playwright con agentes generó y sanó 25 pruebas y descubrió un defecto real. DeepEval evaluó el componente de IA existente y reveló fallas de precisión y exhaustividad. Stagehand, con el modelo `gemini-3-flash-preview`, ejecutó de punta a punta un escenario de tres acciones de IA (barra lateral, auditoría y modo noche) sin respaldos CSS; con un modelo local en CPU resultó demasiado lento (ver `06-IA-EN-TESTING.md`).

4. Conclusiones

1. Los siete requisitos funcionan y están cubiertos por pruebas automatizadas reproducibles; el rediseño no introdujo regresiones frente a la línea base.
2. Las pruebas encontraron defectos reales que la implementación inicial no mostraba (accesibilidad por teclado, reflujo, *debounce*), lo que justifica automatizar axe y las mediciones de reflujo.
3. El componente de IA de alerta de acoso no es confiable por sí solo y requiere revisión humana.
4. La calidad estática es sana (Quality Gate OK, 0 vulnerabilidades en código propio).

5. Recomendaciones

Corregir el detector de acoso (raíces léxicas y contexto); actualizar las 11 pruebas obsoletas; completar la traducción a las demás pantallas; ejecutar el SUS con al menos cinco usuarios; ejecutar el pipeline en el Jenkins institucional y un escaneo ZAP; sustituir `xlsx` por una alternativa mantenida; ejecutar `scripts/create-auditoria-eventos.sql` en Supabase para activar la auditoría en producción.

6. Instalación y ejecución

```
# Back
cd EntreAulas_Back && npm ci && npm test && npm run test:coverage
```

```
# Front
cd EntreAulas_Front && npm ci --legacy-peer-deps && npm test && npm run dev -- -
-port 3001
# E2E (en e2e/, con el front en :3001)
npm ci && env -u ELECTRON_RUN_AS_NODE npx cypress run --e2e --browser chromium -
-spec 'cypress/e2e/**/rediseno-*.cy.ts'
# Playwright
cd ia-testing/playwright && npm ci && npx playwright test tests/rediseno-
# DeepEval
cd ia-testing/deepeval && python3 -m venv .venv && .venv/bin/pip install
deepeval && .venv/bin/python evaluar_ia_final.py --sin-juez
# Docker
docker build -t entreaulas-back:final ../EntreAulas_Back && docker build -t
entreaulas-front:final .
```

Informes y evidencias: `docs/pruebas/final/` (`DEFECTOS.md`, `evidencias/`, matrices `.xlsx`/`.csv`). Repositorios: `github.com/samdg441/EntreAulas_Front` y `…/EntreAulas_Back` (rama `pruebas-final-juliana`, pendiente de subir).

Referencias

Beck, K. (2003). *Test-driven development: By example*. Addison-Wesley.

Brooke, J. (1996). SUS: A "quick and dirty" usability scale. En P. W. Jordan, B. Thomas, B. A. Weerdmeester y I. L. McClelland (Eds.), *Usability evaluation in industry* (pp. 189–194). Taylor & Francis.

International Software Testing Qualifications Board. (2023). *Certified Tester Foundation Level syllabus v4.0*. https://www.istqb.org

McCabe, T. J. (1976). A complexity measure. *IEEE Transactions on Software Engineering, SE-2*(4), 308–320. https://doi.org/10.1109/TSE.1976.233837

Nielsen, J. (1994). *10 usability heuristics for user interface design*. Nielsen Norman Group. https://www.nngroup.com/articles/ten-usability-heuristics/

Open Worldwide Application Security Project. (2021). *OWASP Top 10:2021*. https://owasp.org/Top10/

World Wide Web Consortium. (2018). *Web content accessibility guidelines (WCAG) 2.1*. https://www.w3.org/TR/WCAG21/